

Order Management Demonstration

Title: Order Management System

Objective/Aim:

To write a Java program demonstrating classes, constructors, inheritance, method implementation, and the creation and management of special and normal orders[cite: 11].

System Requirements :

Hardware

- Standard PC/Laptop[cite: 1]
- Intel Core i3/i5 processor or equivalent[cite: 1, 11]
- Minimum 4 GB / 8 GB RAM[cite: 1, 11]
- Minimum 500 MB free storage[cite: 1]
- Keyboard and Monitor[cite: 1]

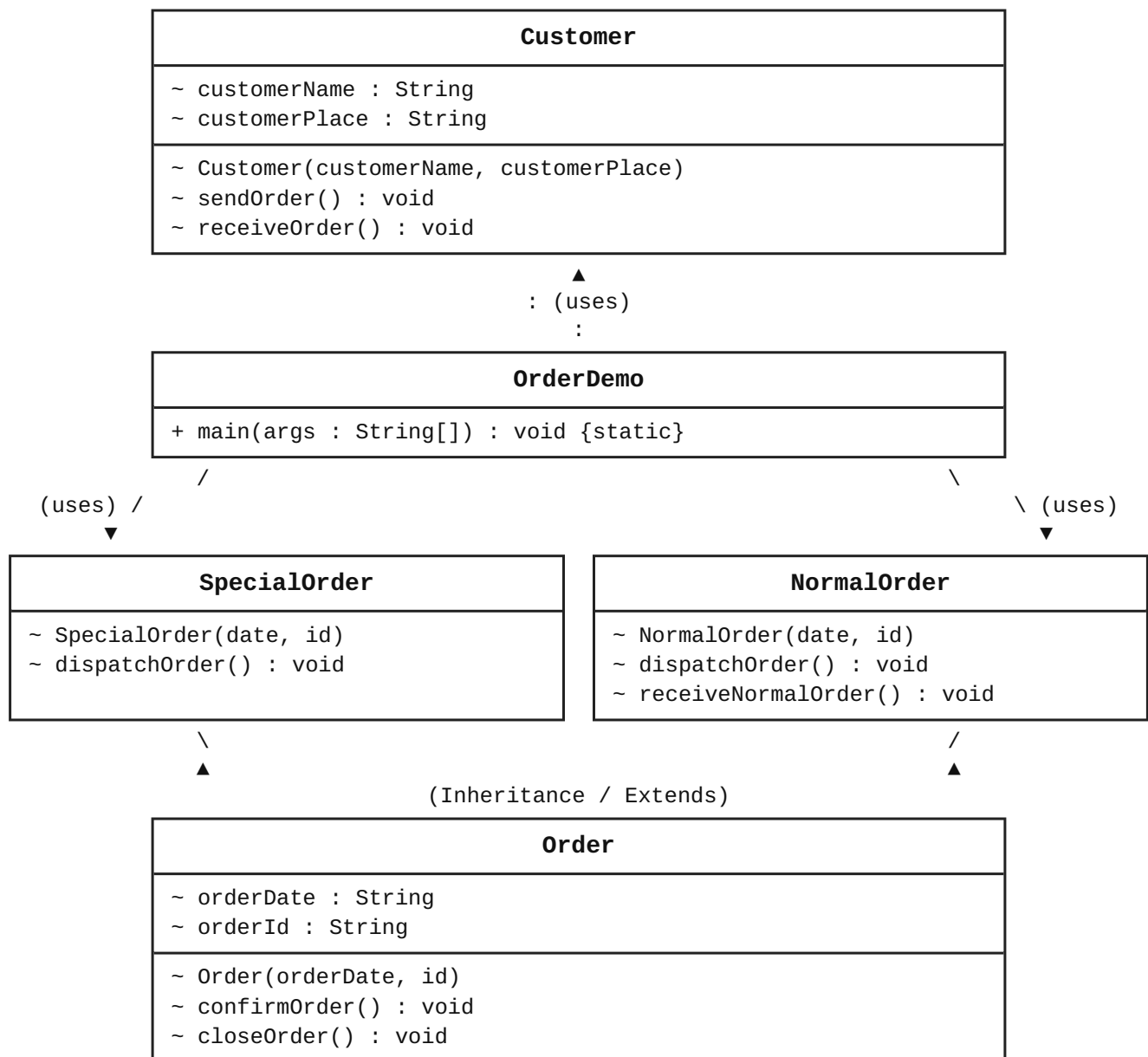
Software

- Operating System: Windows/Linux/macOS[cite: 1, 11]
- Java Development Kit (JDK 17 or JDK 21)[cite: 1, 11]
- Text Editor or IDE such as Eclipse, IntelliJ IDEA, NetBeans, or VS Code[cite: 1, 11]
- Command Prompt/Terminal[cite: 1]

Algorithm / Logic Description:

1. Create a Customer class with customerName and customerPlace as data members[cite: 11, 23].
2. Define sendOrder() and receiveOrder() methods inside the Customer class[cite: 11, 23].
3. Create a base class Order with fields orderDate and orderId[cite: 11, 23].
4. Define confirmOrder() and closeOrder() methods in the Order class[cite: 11, 23].
5. Create SpecialOrder as a derived class of Order, initialize using super(), and define dispatchOrder()[cite: 11, 23].
6. Create NormalOrder as a derived class of Order, initialize using super(), and define dispatchOrder() and receiveNormalOrder()[cite: 11, 23].
7. Create the OrderDemo class containing the main() method[cite: 11, 23].
8. Instantiate a Customer object, a SpecialOrder object, and a NormalOrder object[cite: 11, 23].
9. Display customer details and invoke the order lifecycle methods for both special and normal orders[cite: 11, 24].
10. Stop the program[cite: 2].

UML class Diagram



Relationship: OrderDemo uses Customer, SpecialOrder, and NormalOrder. Both SpecialOrder and NormalOrder inherit from the base class Order[cite: 11].

Source Code:

```
class Customer {
    String customerName;
    String customerPlace;

    Customer(String customerName, String customerPlace) {
        this.customerName = customerName;
        this.customerPlace = customerPlace;
    }

    void sendOrder() {
        System.out.println(customerName + " has sent the order.");
    }
}
```

```

    }

    void receiveOrder() {
        System.out.println(customerName + " has received the order.");
    }
}

class Order {
    String orderDate;
    String orderId;

    Order(String orderDate, String orderId) {
        this.orderDate = orderDate;
        this.orderId = orderId;
    }

    void confirmOrder() {
        System.out.println("Order " + orderId + " confirmed.");
    }

    void closeOrder() {
        System.out.println("Order " + orderId + " closed.");
    }
}

class SpecialOrder extends Order {
    SpecialOrder(String date, String id) {
        super(date, id);
    }

    void dispatchOrder() {
        System.out.println("Special Order " + orderId + " dispatched.");
    }
}

class NormalOrder extends Order {
    NormalOrder(String date, String id) {
        super(date, id);
    }

    void dispatchOrder() {
        System.out.println("Normal Order " + orderId + " dispatched.");
    }

    void receiveNormalOrder() {
        System.out.println("Normal Order " + orderId + " received.");
    }
}

public class OrderDemo {
    public static void main(String[] args) {
        Customer c1 = new Customer("Meghana", "Srikakulam");
        SpecialOrder s1 = new SpecialOrder("11-08-2026", "S101");
        NormalOrder n1 = new NormalOrder("11-08-2026", "N101");

        System.out.println("Customer Information");
        System.out.println("Customer Name: " + c1.customerName);
        System.out.println("Place: " + c1.customerPlace);
    }
}

```

```

        System.out.println("\nSpecial Order Details");
        s1.confirmOrder();
        s1.dispatchOrder();
        s1.closeOrder();

        System.out.println("\nNormal Order Details");
        n1.confirmOrder();
        n1.dispatchOrder();
        n1.receiveNormalOrder();
        n1.closeOrder();

        System.out.println("\nCustomer Activities");
        c1.sendOrder();
        c1.receiveOrder();
    }
}

```

Compilation & Execution Commands:

Save file as: OrderDemo.java[cite: 12]

Compilation: javac OrderDemo.java[cite: 12]

Execution: java OrderDemo[cite: 12]

Output Screenshot:

```

PS C:\Users\mmeen\RAVI> c:; cd 'c:\Users\mmeen\RAVI'; & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\mmeen\AppData\Roaming\Code\User\workspaceStorage\b53c69995fa1d8f837d3efeb869e4899\redhat.java\jdt_ws\RAVI_49fdb1a2\bin' 'Orderdemo'
Customer Information
Customer Name: Ravi
Place: USA

Special Order Details
Order S101 confirmed.
Special Order S101 dispatched.
Order S101 closed.

Normal Order Details
Order N101 confirmed.
Normal Order N101 dispatched.
Normal Order N101 received.
Order N101 closed.

Customer Activities
Ravi has sent the order.
Ravi has received the order.
PS C:\Users\mmeen\RAVI>

```

Result:

The Order Management System program was successfully implemented in Java. The program demonstrates inheritance with the `super ( )` keyword, constructor execution, method overriding, and relationships between customers and orders[cite: 11].